

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\workflows\wf\_8d68d336-288\agent-a40956fc503857d62.jsonl`
- SHA-256: `0d0c5a2635ad9fad0fc42f9c0ce76f37b41b7b6242b2c3376645fdb87708236d`
- Source modified: `2026-07-03T19:50:00+00:00`
- Imported at: `2026-07-08T16:00:15+00:00`
- project: `wf\_8d68d336-288`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-03T19:48:21.203Z)

Repo (my in-progress optimization branch): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/fix-worktree

Key files to Read:

- backend/pipeline/detectors/wasb_infer.py ? the inference loop (DataLoader ? detector.run_tensor ? tracker). Contains the streaming SequentialFrameStore, the _PrefetchIter I just added, build_windows, and the tracker re-run loop.
- backend/pipeline/detectors/native_wasb_runner.py ? builds the wasb_infer.py argv on the GPU box (native runner). Has device/stream_video/batch_size/prefetch_batches config.
- backend/config/models.py (WasbIndexConfig) and backend/config/presets.py (cloud-serving preset).
- deploy/cloudrun/Dockerfile ? the WASB conda env is torch 1.11.0+cu113 (py3.8), a SEPARATE env from the app's torch. The detector shells out to it as a subprocess.

The WASB model itself lives at /opt/models/WASB-SBDT (nttcom/WASB-SBDT) ? you can't read it, but reason from how wasb_infer.py calls build_detector(cfg) / detector.run_tensor(imgs, trans) and build_tracker(cfg).

MEASURED BASELINE (first real L4 cloud run, 2026-07-03): a ~7.5-min 60fps badminton clip took Phase-2 (WASB detection) 727.9s (~0.6x realtime); GPU utilization sat <1% the whole time. Hardware: NVIDIA L4 (Ada sm_89, 23GB), 8 vCPU, 32GB. Streaming mode forces DataLoader num_workers=0 (single-process), so CPU batch assembly (cv2 decode + PIL + ToTensor) serializes with GPU inference.

CRITICAL CONSTRAINT ? the D5/D16 byte-parity guarantee: the repo REQUIRES that the rally/window OUTPUT (the candidate windows and trajectory CSV) stay bit-identical / deterministic across runs and across the disk-frames vs streaming paths. This is asserted by a real parity test. An optimization that changes detector NUMERICS (FP16, TF32, cudnn.benchmark algo selection, NVDEC vs CPU decode) risks changing the output and breaking that guarantee. Batch size and prefetch do NOT change per-frame numerics (same forward, just grouped/overlapped) so they are parity-safe. Already implemented on this branch: indexing.wasb.batch_size + prefetch_batches knobs, the cloud-serving preset opts into batch 64 + prefetch 4.

You are a CUDA/PyTorch performance engineer. Investigate ONE optimization lever in depth by READING the actual code. Lever "batch-loader-tuning":

The batch_size + prefetch_batches knobs are already added (cloud-serving preset = batch 64, prefetch 4). Investigate what the OPTIMAL values likely are on an L4 (23GB) for this model and how to find them empirically, plus SECOND-ORDER loader issues: is the ImageDataset __getitem__ doing redundant work per window (frames_in overlap means adjacent windows re-transform shared frames)? build_windows creates overlapping sliding windows ? is the SAME frame decoded+transformed multiple times across windows? If so, a per-frame transform cache is a bigger win than batching. Quantify the overlap redundancy for frames_in.

Read the named files, reason from the real code, and return the structured finding. Be concrete about code_locations (file:function). Be honest about parity_risk ? the byte-parity guarantee is load-bearing.

2. assistant (2026-07-03T19:48:23.922Z)

I'll investigate the batch-loader-tuning lever by reading the actual code. Let me start with the key files.

3. user (2026-07-03T19:48:29.744Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
```

```

41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63
64     @dataclass
65     class NativeWasbConfig:
66         """Resolved settings for a Linux-native WASB run.
67
68         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71         """
72
73         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74         python_bin: str = (
75             "python" # the `wasb` conda env python (invoked by path, no activate)
76         )
77         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78         sport: str = "badminton"
79         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82         keep_frames: bool = (
83             False # retain the decoded frame cache after success (debug only)
84         )
85         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88         stream_video: bool = True
89         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91         batch_size: int = 0
92         prefetch_batches: int = 0
93
94         @classmethod
95         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96             if isinstance(idx_cfg, dict):
97                 idx_cfg = IndexingConfig(**idx_cfg)

```

```

98         w = idx_cfg.wasb
99         env = os.environ.get
100         # stream_video is tri-state in config: None/absent => native default (stream);
101         an
102         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
103         sv = w.stream_video
104         return cls(
105             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
106             python_bin=env("WASB_PYTHON") or w.python_bin,
107             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
108             sport=w.sport,
109             device=env("WASB_DEVICE") or w.device,
110             scratch_dir=env("WASB_SCRATCH")
111             or env("RALLY_SCRATCH")
112             or env("TMPDIR")
113             or "",
114             timeout_sec=w.timeout_sec,
115             keep_frames=w.keep_frames,
116             stream_video=(True if sv is None else bool(sv)),
117             batch_size=int(w.batch_size or 0),
118             prefetch_batches=int(w.prefetch_batches or 0),
119         )
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
123         docstring."""
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
127             healthcheck;
128             # lazily probed by run_predict if healthcheck was skipped. None = not yet
129             probed.
130             self._cuda_available: Optional[bool] = None
131
132         # --- low-level native exec (the single place tests patch)
133         -----
134         def _run(
135             self, argv: List[str], cwd: Optional[str] = None
136         ) -> subprocess.CompletedProcess:
137             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
138             return subprocess.run(
139                 argv,
140                 cwd=cwd,
141                 capture_output=True,
142                 text=True,
143                 timeout=(self.cfg.timeout_sec or None),
144             )
145
146         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
147         cpu) -----
148         def _probe_cuda(self) -> bool:
149             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
150             Any failure
151             (missing python / timeout / import error) is treated as 'no CUDA'."""
152             try:
153                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
154             except (FileNotFoundError, subprocess.TimeoutExpired):
155                 return False
156             return res.returncode == 0 and "cuda True" in (res.stdout or "")
157
158         def _cuda_is_available(self) -> bool:
159             """CUDA availability, cached across healthcheck + run_predict so we probe at
160             most once."""

```

```

155         if self._cuda_available is None:
156             self._cuda_available = self._probe_cuda()
157         return self._cuda_available
158
159     def _effective_device(self) -> str:
160         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
161         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162         if self.cfg.device == AUTO:
163             return DeviceContext(AUTO, self._cuda_is_available()).effective
164         return self.cfg.device
165
166     def _repo_src(self) -> str:
167         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169     def _copy_infer_script(self) -> bool:
170         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171         repo_src = self._repo_src()
172         try:
173             os.makedirs(repo_src, exist_ok=True)
174             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175             return True
176         except OSError as e:
177             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178             return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182         if path:
183             shutil.rmtree(path, ignore_errors=True)
184
185     def healthcheck(self) -> Tuple[bool, str]:
186         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187         if not self.cfg.weights_path:
188             return False, (
189                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).",
190             )
191         weights = os.path.expanduser(self.cfg.weights_path)
192         if not os.path.exists(weights):
193             return False, f"WASB weights not found at {weights}"
194         repo_src = self._repo_src()
195         if not os.path.isdir(repo_src):
196             return False, (
197                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
198                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).",
199             )
200         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
201         # argparse rc=2 from wasb_infer.py at run time.
202         if self.cfg.device not in VALID_DEVICES:
203             return False, (
204                 f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)}.",
205             )
206         try:
207             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
208         except FileNotFoundError:
209             return (
210                 False,
211                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",

```

```

212         )
213     except subprocess.TimeoutExpired:
214         return False, "native torch healthcheck timed out."
215     out = (res.stdout or "").strip()
216     if res.returncode != 0:
217         return (
218             False,
219             f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
220         )
221     # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
222     self._cuda_available = "cuda True" in out
223     ctx = DeviceContext(self.cfg.device, self._cuda_available)
224     if ctx.cuda_required_but_missing:
225         # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
226         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
227         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
228     if ctx.fell_back:
229         logger.warning(
230             "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
231             "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
232             out,
233         )
234     return True, out
235
236 def run_predict(
237     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
238 ) -> Optional[str]:
239     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
240
241     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
242     runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
243     same downstream contract, so the parity comparison is apples-to-apples.
244     """
245     output_dir = os.path.abspath(output_win_dir)
246     os.makedirs(output_dir, exist_ok=True)
247     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
248     norm = str(video_win_path).replace("\\", "/")
249     stem, _ext = os.path.splitext(os.path.basename(norm))
250     # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
251     key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
252     out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
253
254     if not self._copy_infer_script():
255         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
256         return None
257
258     weights = os.path.expanduser(self.cfg.weights_path)
259     argv = [
260         self.cfg.python_bin,
261         "wasb_infer.py",
262         "--video",
263         os.path.abspath(str(video_win_path)),
264     ]
265     frames_dir: Optional[str] = None
266     if self.cfg.stream_video:
267         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.

```

```

268         argv.append("--stream-video")
269     else:
270         scratch = self.cfg.scratch_dir or output_dir
271         frames_dir = os.path.join(scratch, f"{key}_frames")
272         argv += ["--frames_out_dir", frames_dir]
273     # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
274     # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
275     device = self._effective_device()
276     argv += [
277         "--weights",
278         weights,
279         "--sport",
280         self.cfg.sport,
281         "--device",
282         device,
283         "--out",
284         out_csv,
285     ]
286     # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
its
287     # byte-parity guarantee) is unchanged when the knobs are unset.
288     if self.cfg.batch_size:
289         argv += ["--batch-size", str(self.cfg.batch_size)]
290     if self.cfg.prefetch_batches:
291         argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
292
293     logger.info(
294         "Running native WASB inference (device=%s%s) on %s ...",
295         device,
296         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
297         os.path.basename(norm),
298     )
299     try:
300         res = self._run(argv, cwd=self._repo_src())
301     except FileNotFoundError:
302         logger.error(
303             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
304         )
305         return None
306     except subprocess.TimeoutExpired:
307         logger.error(
308             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
309         )
310         return None
311     if res.returncode != 0:
312         logger.error(
313             "WASB native inference failed:\n%s",
314             (res.stderr or res.stdout or "")[-2000:],
315         )
316         return None
317     if not os.path.exists(out_csv):
318         logger.error(
319             "WASB native finished but expected CSV not found at %s", out_csv
320         )
321         return None
322     logger.info("WASB trajectory CSV: %s", out_csv)
323
324     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
325     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
326     # (Streaming never materializes frames, so there is nothing to drop on that
path.)

```

```

327         if not self.cfg.keep_frames and frames_dir:
328             logger.info(
329                 "Cache hygiene: removing frame cache for %s "
330                 "(set indexing.wasb.keep_frames=true to retain).",
331                 key,
332             )
333             self._rm_rf(frames_dir)
334         return out_csv
335
336     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
337     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
338     trajectory_to_action_windows = staticmethod(
339         TrackNetRunner.trajectory_to_action_windows
340     )
341

```

4. user (2026-07-03T19:48:30.035Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a
18    trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19    resume mid-stream). We exploit that split:
20
21    * The detector pass writes its per-window raw outputs incrementally to
22      ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23      records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24      most ``--flush-every`` windows of GPU work instead of the whole pass.
25    * On restart we replay the cached windows, skip the DataLoader ahead to the first
26      un-cached window, and continue. Once every window is cached, the detector pass
27      is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28      the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29      not the GPU hour.
30
31    The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32    to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33    weights, sport, window size, or frame count change.
34
35    Usage (in WSL, from ~/models/WASB-SBDT/src):
36    python wasb_infer.py --frames_dir /path/frames --weights
37    ../pretrained_weights/wasb_badminton_best.pth.tar \
38    --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
39
40    import argparse
41    import csv
42    import glob
43    import json
44    import logging

```



```

45     import os
46     import os.path as osp
47     from collections import defaultdict
48     from contextlib import contextmanager
49
50     logger = logging.getLogger(__name__)
51
52     CACHE_VERSION = 1
53     _DET_FILE = "det_raw.jsonl"
54     _MANIFEST_FILE = "manifest.json"
55
56
57     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58         """Compose the WASB eval config via Hydra, overriding model/weights/device.
59
60         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
61         preserved), ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
62         ``runner.gpus=[0]``;
63         """
64         from hydra import compose, initialize
65
66         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
67         with initialize(config_path="configs", version_base=None):
68             cfg = compose(
69                 config_name="eval",
70                 overrides=[
71                     f"dataset={sport}",
72                     "model=wasb",
73                     f"detector.model_path={weights}",
74                     f"runner.device={device}",
75                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
76 this box
77             ],
78             )
79         return cfg
80
81     def _frame_id(path: str) -> int:
82         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
83 180)."""
84         stem = osp.splitext(osp.basename(path))[0]
85         digits = "".join(ch for ch in stem if ch.isdigit())
86         return int(digits) if digits else -1
87
88     # ----- #
89     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
90     # ----- #
91     def _cache_paths(cache_dir: str):
92         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
93
94
95     def default_cache_dir(out_csv: str) -> str:
96         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
97         d = osp.dirname(osp.abspath(out_csv))
98         stem = osp.splitext(osp.basename(out_csv))[0]
99         return osp.join(d, stem + "_wasbcache")
100
101
102     def _atomic_write_text(path: str, text: str) -> None:
103         """Write then rename so a crash mid-write can't leave a half-written file."""
104         tmp = path + ".tmp"
105         with open(tmp, "w") as f:

```

```

106         f.write(text)
107         f.flush()
108         os.fsync(f.fileno())
109     os.replace(tmp, path)
110
111
112     def _det_plain(det) -> list:
113         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
114         list."""
115         xy = det["xy"]
116         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
117
118     def _dets_to_tracker(plain_list):
119         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
120         array,
121         the tracker does vector math / np.linalg.norm on it)."""
122         import numpy as np
123         return [
124             {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
125             for p in plain_list
126         ]
127
128
129     def write_manifest(cache_dir: str, manifest: dict) -> None:
130         _, mpath = _cache_paths(cache_dir)
131         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
132
133
134     def read_manifest(cache_dir: str):
135         _, mpath = _cache_paths(cache_dir)
136         if not osp.exists(mpath):
137             return None
138         try:
139             with open(mpath) as f:
140                 return json.load(f)
141         except (json.JSONDecodeError, OSError):
142             return None
143
144
145     def manifest_compatible(
146         manifest: dict,
147         *,
148         frames_dir: str,
149         weights: str,
150         sport: str,
151         frames_in: int,
152         frames_total: int,
153         device: str = "cuda",
154     ) -> bool:
155         """A cache is reusable only if the run that produced it matches this run's inputs.
156
157         ``device`` defaults to ``cuda`` so caches written before device-keying (no
158         ``device``
159         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
160         cache
161         (detections can differ numerically across devices), and vice-versa.
162
163         ``frames_dir`` is the **already-canonical cache key** the caller stored in the
164         manifest
165         (``run()`` passes its ``cache_key`` here, and writes the same value into the
166         manifest):
167         ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
168         ``video:<abspath>``

```

```

164         key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
it here:
165         ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
prepended,
166         so a streaming cache would never match itself and every re-run would recompute
(issue #348).
167         """
168         if not manifest or manifest.get("version") != CACHE_VERSION:
169             return False
170         return (
171             manifest.get("frames_dir") == frames_dir
172             and manifest.get("weights") == weights
173             and manifest.get("sport") == sport
174             and manifest.get("frames_in") == frames_in
175             and manifest.get("frames_total") == frames_total
176             and manifest.get("device", "cuda") == device
177         )
178
179
180     def reset_cache(cache_dir: str) -> None:
181         """Drop any existing cache so the next run starts clean."""
182         det_jsonl, mpath = _cache_paths(cache_dir)
183         for p in (det_jsonl, mpath):
184             if osp.exists(p):
185                 os.remove(p)
186
187
188     def load_and_clean_cache(cache_dir: str):
189         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
190         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
191         prefix so a trailing half-written line from a crash can't corrupt future appends.
192
193         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
194         [x, y, score, scale] candidates (accumulated across every window the frame is in,
195         in window order ? identical to a non-resumed run).
196         """
197         det_jsonl, _ = _cache_paths(cache_dir)
198         det_by_fid = defaultdict(list)
199         valid: list = []
200         if osp.exists(det_jsonl):
201             with open(det_jsonl, "r") as f:
202                 for line in f:
203                     s = line.strip()
204                     if not s:
205                         continue
206                     try:
207                         obj = json.loads(s)
208                     except json.JSONDecodeError:
209                         break # truncated trailing line (crash mid-flush)
210                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
211                         break
212                     valid.append(s)
213                     for fid, plain in obj["f"]:
214                         det_by_fid[int(fid)].extend([list(p) for p in plain])
215                 # Guarantee a clean append boundary by rewriting only the good prefix.
216                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
217         return len(valid), det_by_fid
218
219
220     def _append_windows(fh, objs) -> None:
221         """Append window records as JSONL and durably flush (bounded loss on crash)."""
222         for o in objs:
223             fh.write(json.dumps(o, separators=(",", ":")) + "\n")
224         fh.flush()
225         os.fsync(fh.fileno())

```

```

226
227
228 def _atomic_write_trajectory(out_csv: str, rows) -> None:
229     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
230     tmp = out_csv + ".tmp"
231     with open(tmp, "w", newline="") as f:
232         w = csv.writer(f)
233         w.writerow(["Frame", "Visibility", "X", "Y"])
234         w.writerows(rows)
235         f.flush()
236         os.fsync(f.fileno())
237     os.replace(tmp, out_csv)
238
239
240 # ----- #
241 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
242 # ----- #
243 _STREAM_FRAME_FMT = "{:08d}.png"
244
245
246 def synthetic_frame_path(index: int) -> str:
247     """Stable, index-encoding frame name used by the streaming path.
248
249     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
250     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
251     frames came from disk or from the in-memory stream. The streaming image loader
252     inverts this name back to an index to fetch the decoded frame.
253     """
254     return _STREAM_FRAME_FMT.format(int(index))
255
256
257 def build_windows(frames: list, frames_in: int):
258     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
259     + checkpoint). Pure list math, identical for the disk and streaming paths.
260
261     Returns a list of windows, each a list of ``frames_in`` consecutive paths
262     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
263     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
264     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
265     """
266     return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
267
268
269 class SequentialFrameStore:
270     """In-memory sliding buffer over a forward-only frame reader, sized for the
271     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279     earliest index any not-yet-finished window can still ask for) and decode each source
280     frame exactly once via the forward-only ``reader(index)``.
281     """
282
283     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284         self._reader = reader
285         self._total = int(total)
286         self._window = max(1, int(window))
287         self._buf: dict = {}
288         # On a resumed run the detector's first needed frame is `start_index`; begin
289         # pulling there so the reader can skip (seek past) the already-cached prefix
290         # instead of decoding 0..start_index-1 just to throw them away.

```

```

291         self._next = int(start_index) # next index not yet pulled from the reader
292         self._max_seen = int(start_index) - 1
293         self._floor = int(start_index) # lowest index we still keep (never evict below)
294
295     def get(self, index: int):
296         index = int(index)
297         if index < self._floor:
298             raise KeyError(
299                 f"frame {index} already evicted (below floor {self._floor}); the access
300 "
301                 f"pattern must stay within a window of the highest frame seen"
302             )
303         if index >= self._total:
304             raise KeyError(f"frame {index} out of range (total {self._total})")
305         # Pull forward from the reader until the requested index is buffered.
306         while self._next <= index:
307             self._buf[self._next] = self._reader(self._next)
308             self._next += 1
309         if index > self._max_seen:
310             self._max_seen = index
311         # Evict frames older than the earliest a still-running window could re-request:
312         # once we've seen index `m`, no future window starts before `m - (window-1)`.
313         new_floor = max(self._floor, self._max_seen - (self._window - 1))
314         for k in range(self._floor, new_floor):
315             self._buf.pop(k, None)
316         self._floor = new_floor
317         return self._buf[index]
318
319     def _index_from_synthetic_path(path: str) -> int:
320         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
321
322         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
323         in lockstep with the on-disk naming scheme.
324         """
325         return _frame_id(path)
326
327     def _bgr_to_pil_rgb(frame_bgr):
328         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
329         EXACT
330         PIL RGB image WASB's disk path produces.
331
332         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
333         Image.open(PNG).convert('RGB')``;
334         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
335         COLOR_BGR2RGB)``
336         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
337         streamed
338         frame indistinguishable from the on-disk one to everything downstream.
339         """
340         import cv2
341         from PIL import Image
342
343         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
344
345     def _make_streamed_read_image(store, real_read_image):
346         """Build the ``read_image`` replacement used during a streaming detector pass.
347
348         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
349         image,
350         matching ``read_image``'s real return type); any path that doesn't parse to an index
351         falls
352         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-

```

```

testable
349     without the WSL-only ``dataloaders`` package.
350     """
351
352     def _read_image(path, *args, **kwargs):
353         idx = _index_from_synthetic_path(path)
354         if idx < 0:
355             return real_read_image(path, *args, **kwargs)
356         return _bgr_to_pil_rgb(store.get(idx))
357
358     return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
366         decoded
367         frames during the detector pass, byte-for-byte as it would over real PNGs.
368
369         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
370         ``cv2.imread``. ``read_image`` (``utils.utils``) does
371         ``Image.open(path).convert('RGB')``,
372         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
373         feed it a
374         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
375         byte-identical to the disk round-trip, and the shim never touches the filesystem so
376         the
377         ``osp.exists`` guard is sidestepped for synthetic stream paths.
378
379         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
380         actually
381         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
382         in that
383         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
384         imported
385         reference).
386
387         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
388         wrapped
389         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
390         on a
391         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
392         The
393         original reader is restored on exit.
394         """
395         from dataloaders import dataset_loader as _dl
396
397         store = SequentialFrameStore(
398             frame_loader, total, window=window, start_index=start_index
399         )
400         real_read_image = _dl.read_image
401         # Rebind read_image in the consuming module for the duration of the detector pass;
402         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
403         B010-clean.)
404         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
405         ignore[assignment]
406         try:
407             yield store
408         finally:
409             _dl.read_image = real_read_image # type: ignore[assignment]
410
411     # ----- #

```

```

401     # Inference
402     # ----- #
403     class _PrefetchIter:
404         """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).
405
406         In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
407         the cv2.imread shim live in this process and cannot be pickled to workers), so the
408         CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
409         measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
410
411         A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
412         release the GIL for the heavy parts, the store's forward-only access pattern is
413         preserved (one consumer thread iterating sequentially), and the bounded queue caps
414         memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
415
416         _END = object()
417
418         def __init__(self, src, depth: int = 2):
419             import queue
420             import threading
421
422             self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
423             self._exc: "BaseException | None" = None
424
425             def _fill() -> None:
426                 try:
427                     for item in src:
428                         self._q.put(item)
429                     except BaseException as e: # noqa: BLE001 - surfaced in __next__
430                         self._exc = e
431                     finally:
432                         self._q.put(self._END)
433
434             self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
435             self._t.start()
436
437         def __iter__(self):
438             return self
439
440         def __next__(self):
441             item = self._q.get()
442             if item is self._END:
443                 if self._exc is not None:
444                     raise self._exc
445                 raise StopIteration
446             return item
447
448     def run(
449         frames,
450         weights: str,
451         out_csv: str,
452         sport: str = "badminton",
453         limit: int = 0,
454         batch_size: int = 8,
455         num_workers: int = 4,
456         log_every_batches: int = 50,
457         cache_dir: "str | None" = None,
458         flush_every: int = 1000,
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",

```

```

463     prefetch_batches: int = 0,
464 ) -> str:
465     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467     Two equivalent ways to supply pixels (output is bit-identical between them):
468
469     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470       globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
471       ``cv2.imread``.
472     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
473       frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
474       pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
475       reader, which returns a PIL RGB image) over the DataLoader pass so every other
476       line of
477       ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
478       identical
479       to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
480       ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
481       ``num_workers=0``
482       (the in-process shim + buffer can't cross worker processes).
483
484     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
485     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
486     after reboot still matches.
487     """
488     import time
489
490     import torch
491     from dataloaders import build_img_transforms, build_seq_transforms
492     from dataloaders.dataset_loader import ImageDataset
493     from torch.utils.data import DataLoader
494
495     # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
496     the
497     # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
498     suppresses
499     # the mypy false positive where the main env resolves `trackers` to the Roboflow
500     package
501     # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
502     separate
503     # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
504     dep.
505     from trackers import build_tracker # type: ignore[attr-defined]
506     from utils import Center
507
508     from detectors import build_detector
509
510     streaming = frame_loader is not None
511
512     cfg = _build_cfg(weights, sport, device)
513     frames_in = int(cfg["model"]["frames_in"])
514     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
515     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
516
517     if streaming:
518         # `frames` is already the ordered list of (synthetic) frame paths.
519         if not isinstance(frames, (list, tuple)):
520             raise SystemExit(
521                 "streaming mode requires `frames` to be a list of frame paths"
522             )
523         frames = list(frames)
524         frames_dir = source_key or "stream"
525     else:
526         frames_dir = frames
527         frames = sorted(

```



```

519         glob.glob(osp.join(frames_dir, "*.png"))
520         + glob.glob(osp.join(frames_dir, "*.jpg"))
521     )
522     if limit:
523         frames = frames[:limit]
524     if len(frames) < frames_in:
525         where = frames_dir if not streaming else "video stream"
526         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
529     samples = []
530     for window in build_windows(frames, frames_in):
531         annos = [
532             {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
533             for p in window
534         ]
535         samples.append({"frames": window, "annos": annos})
536     windows_total = len(samples)
537
538     # ---- cache setup / resume decision ----- #
539     if cache_dir is None:
540         cache_dir = default_cache_dir(out_csv)
541     os.makedirs(cache_dir, exist_ok=True)
542     det_jsonl, _ = _cache_paths(cache_dir)
543     cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
544     manifest = None if fresh else read_manifest(cache_dir)
545     resume = manifest_compatible(
546         manifest or {},
547         frames_dir=cache_key,
548         weights=weights,
549         sport=sport,
550         frames_in=frames_in,
551         frames_total=len(frames),
552         device=device,
553     )
554     if resume:
555         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
556         logger.info(
557             f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
558         )
559     else:
560         if manifest is not None:
561             logger.info("cache incompatible with this run -> starting fresh")
562             reset_cache(cache_dir)
563             windows_done, det_by_fid = 0, defaultdict(list)
564
565     base_manifest = {
566         "version": CACHE_VERSION,
567         "frames_dir": cache_key,
568         "weights": weights,
569         "sport": sport,
570         "frames_in": frames_in,
571         "frames_total": len(frames),
572         "device": device,
573         "windows_total": windows_total,
574         "windows_done": windows_done,
575     }
576     write_manifest(cache_dir, base_manifest)
577
578     # ---- detector pass over the un-cached windows ----- #
579     remaining = samples[windows_done:]
580     if remaining:
581         _, transform_test = build_img_transforms(cfg)

```

```

582         try:
583             _, seq_transform_test = build_seq_transforms(cfg)
584         except Exception:
585             seq_transform_test = None
586     ds = ImageDataset(
587         cfg,
588         remaining,
589         input_wh,
590         output_wh,
591         transform=transform_test,
592         seq_transform=seq_transform_test,
593         is_train=False,
594     )
595     # Streaming forces single-process loading: the in-memory frame store + the
596     # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597     loader_workers = 0 if streaming else num_workers
598     loader = DataLoader(
599         ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600     )
601     detector = build_detector(cfg)
602
603     from contextlib import ExitStack
604
605     t0 = time.time()
606     done = windows_done
607     win_idx = windows_done
608     log_every = max(1, log_every_batches)
609     flush_n = max(1, flush_every)
610     pending = []
611     logger.info(
612         f"detecting on {len(remaining)} remaining windows "
613         f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614         f"source={'video-stream' if streaming else 'frames-dir'})..."
615     )
616     det_f = open(det_jsonl, "a")
617     try:
618         with ExitStack() as stack:
619             stack.enter_context(torch.no_grad())
620             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621             # in-memory decoded frame for that synthetic path; every other line of
622             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
623             # The detector's first needed frame is `windows_done` (window i starts
at
624             # frame i), so prime the store there for a resume.
625             if streaming:
626                 stack.enter_context(
627                     _streaming_read_image_patch(
628                         frame_loader,
629                         len(frames),
630                         window=frames_in,
631                         start_index=windows_done,
632                     )
633                 )
634             # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635             # a multi-worker DataLoader already overlaps via its worker processes).
636             batches = (
637                 _PrefetchIter(loader, depth=prefetch_batches)
638                 if prefetch_batches and loader_workers == 0
639                 else loader
640             )
641             for bi, batch in enumerate(batches):

```

```

642         imgs, _hms, trans = batch[0], batch[1], batch[2]
643         img_paths = [
644             list(t) for t in batch[-1]
645         ] # [frame_in][batch] -> path
646         batch_results, _ = detector.run_tensor(imgs, trans)
647         nb = imgs.shape[0]
648         for ib in range(nb):
649             frames_payload = []
650             for ie in sorted(batch_results[ib].keys()):
651                 p = img_paths[ie][ib]
652                 fid = _frame_id(p)
653                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                 frames_payload.append([fid, plain])
655                 det_by_fid[fid].extend(plain)
656             pending.append({"w": win_idx, "f": frames_payload})
657             win_idx += 1
658         done += nb
659         if len(pending) >= flush_n or done >= windows_total:
660             _append_windows(det_f, pending)
661             pending = []
662             base_manifest["windows_done"] = done
663             write_manifest(cache_dir, base_manifest)
664         if (bi + 1) % log_every == 0 or done >= windows_total:
665             el = time.time() - t0
666             new_done = done - windows_done
667             rate = new_done / el if el > 0 else 0.0
668             eta = (windows_total - done) / rate if rate > 0 else 0.0
669             logger.info(
670                 f"{done}/{windows_total} "
671                 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
672                 f"elapsed {el:.0f}s | ETA {eta:.0f}s"
673             )
674         if pending:
675             _append_windows(det_f, pending)
676             base_manifest["windows_done"] = done
677             write_manifest(cache_dir, base_manifest)
678         finally:
679             det_f.close()
680             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
681     else:
682         logger.info("detector cache already complete -> tracker-only re-run")
683
684     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
685     tracker = build_tracker(cfg)
686     tracker.refresh()
687     rows = []
688     for fid in sorted(det_by_fid.keys()):
689         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
690         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
691
692     _atomic_write_trajectory(out_csv, rows)
693     visible = sum(1 for _, v, *_ in rows if v)
694     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
695     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
696     return out_csv
697
698
699 def extract_frames(video_path: str, frames_dir: str) -> str:
700     """Decode a video to PNG frames named by source frame index (00000000.png ...).
701
702     Idempotent: if the dir already holds the expected number of frames (matching the
703     container's reported frame count) we skip re-decoding ? re-extraction after a
704     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
705     extracting with ffmpeg on the host and passing --frames_dir.

```

```

706     """
707     import cv2
708
709     os.makedirs(frames_dir, exist_ok=True)
710     cap = cv2.VideoCapture(video_path)
711     if not cap.isOpened():
712         raise SystemExit(f"cannot open video: {video_path}")
713     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
714     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
715     if expected > 0 and existing >= expected:
716         cap.release()
717         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
718         return frames_dir
719     i = 0
720     while True:
721         ok, frame = cap.read()
722         if not ok:
723             break
724         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
725         i += 1
726     cap.release()
727     logger.info(f"extracted {i} frames -> {frames_dir}")
728     return frames_dir
729
730
731     def _probe_frame_count(video_path: str) -> int:
732         """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
733         import cv2
734
735         cap = cv2.VideoCapture(video_path)
736         if not cap.isOpened():
737             raise SystemExit(f"cannot open video: {video_path}")
738         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
739         cap.release()
740         return n
741
742
743     def make_video_frame_loader(video_path: str):
744         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
745
746         ``frame_loader(index)`` -> BGR uint8 ndarray`` decodes the video forward-only with one
747         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748         detector's sliding-window order); it reads sequentially and only uses a frame-seek
749         to
750         skip forward when a *resume* starts past the current position. Each ``cap.read()``
751         returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
752         would
753         yield (PNG is lossless), so the detector sees identical pixels to the disk path.
754
755         Returns the count hint from the container so the caller can build the frame list;
756         ``release`` closes the capture.
757         """
758         import cv2
759
760         cap = cv2.VideoCapture(video_path)
761         if not cap.isOpened():
762             raise SystemExit(f"cannot open video: {video_path}")
763         count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
764         state = {"pos": 0} # next index cap.read() will return
765
766         def frame_loader(index: int):
767             index = int(index)
768             if index < state["pos"]:
769                 raise RuntimeError(

```

```

768             f"streaming decode is forward-only; got index {index} after
{state['pos']]}"
769         )
770         if index > state["pos"]:
771             # Forward skip only happens when resuming past cached windows. Seek once.
772             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
773             state["pos"] = index
774             ok, frame = cap.read()
775             if not ok:
776                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
777             state["pos"] += 1
778             return frame
779
780     def release():
781         cap.release()
782
783     return frame_loader, count_hint, release
784
785
786 def run_video_streaming(
787     video_path: str,
788     weights: str,
789     out_csv: str,
790     sport: str = "badminton",
791     limit: int = 0,
792     batch_size: int = 8,
793     log_every_batches: int = 50,
794     cache_dir: "str | None" = None,
795     flush_every: int = 1000,
796     fresh: bool = False,
797     device: str = "cuda",
798     prefetch_batches: int = 0,
799 ) -> str:
800     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
801     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
802     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
803     ``run``'s docstring for why.
804     """
805     frame_loader, count_hint, release = make_video_frame_loader(video_path)
806     try:
807         if count_hint <= 0:
808             raise SystemExit(
809                 f"could not determine frame count for {video_path}; cannot stream "
810                 f"(fall back to frame extraction with --frames_dir)"
811             )
812         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
813         source_key = "video:" + osp.abspath(video_path)
814         return run(
815             frame_list,
816             weights,
817             out_csv,
818             sport,
819             limit,
820             batch_size=batch_size,
821             num_workers=0,
822             log_every_batches=log_every_batches,
823             cache_dir=cache_dir,
824             flush_every=flush_every,
825             fresh=fresh,
826             frame_loader=frame_loader,
827             source_key=source_key,
828             device=device,
829             prefetch_batches=prefetch_batches,
830         )
831     finally:

```

```

832         release()
833
834
835 def main():
836     ap = argparse.ArgumentParser()
837     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
838     ap.add_argument(
839         "--video", help="video file; frames are extracted internally (in WSL)"
840     )
841     ap.add_argument(
842         "--frames_out_dir", help="where to extract frames when --video is used"
843     )
844     ap.add_argument(
845         "--stream-video",
846         action="store_true",
847         help="FAST PATH: decode --video on the fly and feed frames straight to "
848         "the detector (no PNG extraction). Output-equivalent to the disk path.",
849     )
850     ap.add_argument("--weights", required=True)
851     ap.add_argument("--out", required=True)
852     ap.add_argument("--sport", default="badminton")
853     ap.add_argument(
854         "--device",
855         default="cuda",
856         choices=["cuda", "mps", "cpu"],
857         help="torch device for the detector (default cuda = the WSL parity path)",
858     )
859     ap.add_argument(
860         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
861     )
862     ap.add_argument("--batch-size", type=int, default=8)
863     ap.add_argument("--num-workers", type=int, default=4)
864     ap.add_argument(
865         "--prefetch-batches",
866         type=int,
867         default=0,
868         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
869         "bounded producer thread of this queue depth (0 = off, the parity default)",
870     )
871     ap.add_argument(
872         "--log-every-batches", type=int, default=50, help="progress log cadence"
873     )
874     ap.add_argument(
875         "--cache-dir",
876         default=None,
877         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
878     )
879     ap.add_argument(
880         "--flush-every",
881         type=int,
882         default=1000,
883         help="fsync the detector cache every N windows (bounds reboot loss)",
884     )
885     ap.add_argument(
886         "--fresh", action="store_true", help="ignore any existing cache and start over"
887     )
888     args = ap.parse_args()
889
890     if args.stream_video:
891         if not args.video:
892             raise SystemExit("--stream-video requires --video")
893         run_video_streaming(
894             args.video,
895             args.weights,
896             args.out,

```

```

897         args.sport,
898         args.limit,
899         batch_size=args.batch_size,
900         log_every_batches=args.log_every_batches,
901         cache_dir=args.cache_dir,
902         flush_every=args.flush_every,
903         fresh=args.fresh,
904         device=args.device,
905         prefetch_batches=args.prefetch_batches,
906     )
907     return
908
909     frames_dir = args.frames_dir
910     if args.video:
911         frames_dir = extract_frames(
912             args.video, args.frames_out_dir or (args.video + "_frames")
913         )
914     if not frames_dir:
915         raise SystemExit("provide --frames_dir or --video")
916     run(
917         frames_dir,
918         args.weights,
919         args.out,
920         args.sport,
921         args.limit,
922         batch_size=args.batch_size,
923         num_workers=args.num_workers,
924         log_every_batches=args.log_every_batches,
925         cache_dir=args.cache_dir,
926         flush_every=args.flush_every,
927         fresh=args.fresh,
928         device=args.device,
929     )
930
931
932 if __name__ == "__main__":
933     main()
934

```

5. assistant (2026-07-03T19:48:32.357Z)

Now let me look at the config models and presets to understand the current knob values and defaults.

6. user (2026-07-03T19:48:34.982Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])

```

```

20     court_green_upper: list[int] = Field(default=[85, 255, 255])
21     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."
39             ),
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45     class TrackNetConfig(BaseModel):
46         wsl_distro: str = "Ubuntu"
47         repo_dir: str = "~/models/TrackNetV4"
48         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
49         conda_env: str = "TrackNetV4"
50         weights_path: str = ""
51         queue_length: int = 5
52         stage_in_wsl: bool = True
53         wsl_stage_dir: str = "~/clips"
54         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
55         velocity_threshold: float = 0.02
56         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
57         # decoded frame cache) are deleted automatically ? they are pure intermediates,
58         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
59         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
60         keep_frames: bool = False
61         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
62         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
63         # not warns).
64         # 0 = no extra floor (the headroom guard still applies).
65         min_free_gb: float = 0.0
66
67     class WasbIndexConfig(BaseModel):
68         """Typed config for the live WASB shuttle substrate (indexing.wasb).
69
70         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
71         these knobs actually take effect ? previously this whole block was silently dropped
72         because nested config sections defaulted to extra='ignore'.
73         """
74
75         wsl_distro: str = "Ubuntu"
76         repo_dir: str = "~/models/WASB-SBDT"
77         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
78         conda_env: str = "wasb"
79         weights_path: str = (
80             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
81         )
82

```



```

83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101    # 0 = no extra floor (the headroom guard still applies).
102    min_free_gb: float = 0.0
103    # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104    # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105    # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106    # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107    # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108    # Tri-state:
109    # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110    # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111    # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112    stream_video: Optional[bool] = None
113    # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114    # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115    # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117    # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118    batch_size: Optional[int] = None
119    # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120    # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121    prefetch_batches: Optional[int] = None
122
123
124    class FusionConfig(BaseModel):
125        """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
126
127        Every default reproduces control behaviour: the fusion segmenter persists the
128        shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is

```

```

129     true ? the person-cue features, but it does NOT gate the served handoff unless a
130     threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court
mask
131     is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
132     supplied ? off-court persons (spectators / adjacent courts) are excluded.
133     ""
134
135     # Which trajectory substrate seeds the windows (shuttle stays primary).
136     substrate: Literal["wasb", "tracknet"] = "wasb"
137     # Windowing velocity threshold for the fusion family (the engine reads
138     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
139     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
140     velocity_threshold: float = 0.02
141     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
142     # enabled ? so the default path never imports torch / touches the GPU.
143     cue_flags: dict[str, bool] = Field(default_factory=dict)
144     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
145     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
146     min_crossings: int = Field(default=0, ge=0)
147     # Served Gate B: when the person cue is on, drop windows with median in-court
players
148     # < this. 0 = OFF.
149     min_players: int = Field(default=0, ge=0)
150     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
151     court_polygon: Optional[list[list[float]]] = None
152     # Net line for the person two-sided-motion split (person features only ? the shuttle
153     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
154     net_axis: Literal["x", "y"] = "y"
155     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
156
157
158     class IndexingConfig(BaseModel):
159         default_segmenter: str = "yolo_hybrid"
160         use_gpu: bool = True
161         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
162         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
163         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
164         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
165         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
166         detector_impl: str = "auto"
167         motion_threshold: float = 0.08
168         min_segment_duration: float = 3.0
169         dead_time_merge_gap: float = 2.0
170         chunk_duration: float = 90.0
171         chunk_overlap: float = 10.0
172         detector_model: str = "fasterrcnn_resnet50_fpn"
173         detector_score_threshold: float = 0.5
174         yolo_velocity_threshold: float = 0.15
175         yolo_frame_skip: int = 3
176         yolo_tracker: str = "bytetrack.yaml"
177         yolo_prefetch_workers: int = 2
178         min_action_window_duration: float = 2.0
179         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
=a window
180         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
181         # `window_min_split_gap` s ? the most likely rally boundary), so one window can't
swallow
182         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
183         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.

```

```

Measured n=13
184      # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
185      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
186      max_window_duration: float = 6.0
187      window_min_split_gap: float = (
188          0.5 # min internal gap (s) that qualifies as a split point
189      )
190      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
191      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
192      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
193      # Only cuts (recall can't drop). OFF by default until measured/promoted.
194      window_hard_cap: bool = False
195      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
196      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
197      # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
198      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
199      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
200      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
201      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
202      window_inactivity_end: float = 1.5
203      inactivity_rally_thresh: float = 0.20
204      inactivity_min_density: float = 0.30
205      # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
206      # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
207      # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
208      # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
209      # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
210      # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
211      # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
212      # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
213      # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
214      inactivity_temporal_density: bool = True
215      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
216      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
217      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
218      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
219      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
220      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
221      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER

```

R4 and only

```
222     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
223     window_blob_fallback: bool = False
224     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
225     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
226     # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
227     # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
228     window_blob_factor: float = 4.0
229     log_yolo_candidates: bool = True
230     store_yolo_candidates: bool = True
231     ai_handoff_padding: float = 2.0
232     ai_max_workers: int = 5
233     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
234     # chunks of a high-bitrate (4K) source blow past the provider upload cap
235     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
236     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
237     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
238     # Matches gemini_refine's --clip-scale-width default.
239     ai_chunk_scale_width: int = 1280
240     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
241     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
242     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
243     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
244     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
245     skip_ai_handoff: bool = False
246     # Optional debug knob: trim every video to the first N seconds before processing.
247     debug_duration: Optional[float] = None
248
249     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
250     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
251     fusion: FusionConfig = Field(default_factory=FusionConfig)
252
253     @field_validator("default_segmenter")
254     @classmethod
255     def segmenter_must_be_registered(cls, v: str) -> str:
256         # Lazy import: the segmenter registry imports modules that may import
257         # config, so we resolve it at validation time rather than at module load.
258         from backend.pipeline.segmenters.base import segmenter_registry
259
260         if segmenter_registry.get(v) is None:
261             available = sorted(segmenter_registry.list_available().keys())
262             raise ValueError(
263                 f"Segmenter '{v}' is not registered. Available: {'', '.join(available)}"
264             )
265         return v
266
267     @model_validator(mode="after")
268     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
269         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
270         # step makes `while t < duration: t += step` loop forever, growing memory before
any
271         # GPU work. Reject the bad config at load time instead.
272         if self.chunk_overlap >= self.chunk_duration:
273             raise ValueError(
274                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
```

```

275         f"({self.chunk_duration}); otherwise chunking never advances."
276     )
277     return self
278
279
280 class OutputConfig(BaseModel):
281     default_highlights_name: str = "highlights.mp4"
282     db_name: str = "sports_indexer.db"
283     # Directory where the DB, highlight reels, and processed clips are written.
284     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
285     output_dir: str = "output"
286
287
288 class WatermarkConfig(BaseModel):
289     """Branding overlay burned into each highlight clip during the per-clip re-encode
290     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
291     (under `stitching.watermark` in config.json) to turn it off. The text is fully
292     configurable. The concat stage stays stream-copy (-c copy)."""
293
294     enabled: bool = True
295     text: str = "Generated by Khelsutra.guru"
296     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
297     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
298     font: str = "Sans" # fontconfig family used when font_path is empty
299     font_size_ratio: float = Field(
300         default=0.035, gt=0.0, le=0.5
301     ) # text height as a fraction of frame height
302     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
303     box: bool = True # faint backing box behind the text for legibility
304     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
305     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
306         "bottom-right"
307     )
308
309
310 class StitchingConfig(BaseModel):
311     begin_padding: float = 0.5
312     end_padding: float = 0.5
313     use_cache: bool = False
314     min_shot_count: int = 1
315     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
316     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
317     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
318     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
319     # local detector over-fires and its false positives are mostly SHORT (motion blips,
320     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
321     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
322     min_rally_duration: float = Field(default=0.0, ge=0.0)
323     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
324
325
326 class LoggingConfig(BaseModel):
327     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
328
329     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
330     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
331     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
332     # them to WARNING; set true to restore full chatter for request-flow debugging.
333     verbose_http: bool = False
334

```

```

335
336 class SecurityConfig(BaseModel):
337     # When set, /api/process video_path must resolve under this directory (hosted/tenant
338     # mode). Default None preserves the single-user local 'any local file' workflow.
339     allowed_video_root: Optional[str] = None
340
341     # --- Chunked upload (B2, PR-2) ---
342     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
343     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
344     # contain upload_dir or /api/process will reject the uploaded paths.
345     upload_dir: str = "output/uploads"
346     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
347     # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
348     max_upload_mb: int = 4096
349     max_part_mb: int = 95
350     allowed_upload_extensions: List[str] = ["mp4", "mov"]
351
352     # --- M9 edge auth (Cloudflare Access, D9) ---
353     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
354     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
355     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
356     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
357     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
358     # application's audience tag. NO secrets here ? these are public identifiers.
359     edge_auth_enabled: bool = False
360     cf_team_domain: str = (
361         "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
362     )
363     cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
364     # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
365     # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
366     # OFF ? Cf-Access is a header, not a cookie.)
367
368
369 class StorageConfig(BaseModel):
370     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
371     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
372     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
373     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
374     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
375     (boto3/google auth chains supply credentials). See ``backend/storage/``. """
376
377     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
378     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
379     output_root: str = ""
380     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
381     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
382     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
383     # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"

```

```

384         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
    ``input_backend``
385         # (``"gcs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the
input
386         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
387         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
388         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
389         input_root: str = ""
390         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
391         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
392         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
393         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
394         # the legacy flat layout is used unchanged.
395         single_folder_per_video: bool = False
396         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
397         workspace_root: str = ""
398         # Cloud namespace prefix under the root so per-video folders sit tidily beside
399         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
400         workspace_prefix: str = "workspaces"
401         local: dict[str, Any] = Field(default_factory=dict)
402         s3: dict[str, Any] = Field(default_factory=dict)
403         gcs: dict[str, Any] = Field(default_factory=dict)
404         gdrive: dict[str, Any] = Field(default_factory=dict)
405         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
406         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
407         gdrive_api: dict[str, Any] = Field(default_factory=dict)
408
409
410     class DeploymentConfig(BaseModel):
411         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
412         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
413
414         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
415         behaves exactly as before this section existed. A profile is opt-in (``config.json``
416         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
417         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
418         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
419         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
420
421         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
422
423         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
424         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
425         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
426         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
427         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
428
429
430     class BillingConfig(BaseModel):
431         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
432

```

```

433     **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
434     today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
435     ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
436     version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
437     inserts a real, calibrated version and points ``pricebook_version`` at it.
438
439     ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
440     wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
441     pricebook + the recording seam; usage emission (the metering hooks) is a separate
step.
442
443     enabled: bool = False
444     pricebook_version: str = (
445         "placeholder-v0" # the seeded $0 version; owner flips to a real one
446     )
447     fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
448     margin: float = 0.0 # resale markup (?0); 0 = bill at cost
449
450
451     class FlywheelConfig(BaseModel):
452         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
453         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
454         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
455         the owned model climbs toward the D11 ship floor.
456
457         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
458         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
459         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
460         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-row).
461         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
462         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
463         owner/counsel/GPU scope. Pairs with M9-consent.
464
465         enabled: bool = False
466         workspace_prefix: str = (
467             "flywheel" # per-video workspace prefix for captured artifacts
468         )
469         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
470         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
471
472
473     class AppConfig(BaseModel):
474         """Root configuration object.
475
476         All runtime configuration should be accessed via an instance of this class
477         (or the loader) rather than raw dict lookups.
478
479
480         sport: Sport = "badminton"
481         ai_provider: str = "gemini"

```



```

482     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
483     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
484     ai_model: str = "gemini-flash-latest"
485
486     validation: ValidationConfig = Field(default_factory=ValidationConfig)
487     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
488     output: OutputConfig = Field(default_factory=OutputConfig)
489     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
490     security: SecurityConfig = Field(default_factory=SecurityConfig)
491     logging: LoggingConfig = Field(default_factory=LoggingConfig)
492     storage: StorageConfig = Field(default_factory=StorageConfig)
493     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
494     billing: BillingConfig = Field(default_factory=BillingConfig)
495     flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
496
497     # Reject unknown keys ? typed config migration is complete (D3).
498     model_config = {
499         "extra": "forbid",
500         "validate_assignment": True,
501     }
502
503
504     # Convenience type alias for callers that want to annotate
505     Config = AppConfig
506

```

7. user (2026-07-03T19:48:35.366Z)

```

1     """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3     A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4     config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5     knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7     Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn

```

```

31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 "wasb": {"batch_size": 64, "prefetch_batches": 4},
61             },
62             "storage": {"backend": "gcs"},
63         },
64         "cpu-mock": {
65             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
67             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
68             "storage": {"backend": "local"},
69         },
70     }
71
72
73     def is_known_profile(profile: str) -> bool:
74         """True for a non-empty, recognised profile name."""
75         return bool(profile) and profile in PROFILE_PRESETS
76
77
78     def preset_for(profile: str) -> dict[str, Any]:
79         """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
80         module-level table), or ``{}`` for ``""`` / an unknown profile."""
81         return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82
83
84     def _truthy(val: Optional[str]) -> bool:
85         """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""

```

```

86         if val is None:
87             return False
88         return val.strip().lower() in {"1", "true", "yes", "on"}
89
90
91     def _cuda_available() -> bool:
92         """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
93         ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
94         try:
95             # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
96             # and optional). Only reached when a caller opts into probe_cuda.
97             import torch
98
99             return bool(torch.cuda.is_available())
100     except Exception:
101         return False
102
103
104     def probe_cuda_available() -> bool:
105         """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
106
107         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
108         that *wants* the GPU heads-up can pass the result in without the startup module
importing
109         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
110         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
111         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
112         error that could false-fail."""
113         return _cuda_available()
114
115
116     def suggest_profile(
117         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
118     ) -> str:
119         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
120
121         Signals, in priority order:
122
123         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
124         - ``CI`` truthy                            ? ``"cpu-mock"``
125         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
126         - otherwise                               ? ``""`` (no strong suggestion)
127
128         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
129         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
130         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
131
132         """
133         e = os.environ if env is None else env
134         if e.get("K_SERVICE"):
135             return "cloud-serving"
136         if _truthy(e.get("CI")):
137             return "cpu-mock"
138         if probe_cuda and _cuda_available():
139             return "truly-local"
140         return ""

```

8. assistant (2026-07-03T19:48:37.931Z)

Now let me look at the Dockerfile to understand the WASB env constraints, and check the WASB config to find the actual `frames\_in` value for badminton.

9. user (2026-07-03T19:48:41.024Z)

```

1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
33     # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
34     # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
35     # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
36     # MAMBA_ROOT_PREFIX=$CONDA_DIR (= /opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
37     # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
38     ENV MAMBA_ROOT_PREFIX=/opt/conda
39     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \

```

```

40         | tar -xj -C /usr/local bin/micromamba \
41     && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
42     && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
43     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
44         torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
45     && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
46     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
47         hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
48
49     # --- Main application env ---
50     WORKDIR /app
51     COPY pyproject.toml README.md ./
52     COPY requirements-trackers.txt ./
53     COPY backend ./backend
54     COPY training ./training
55     # Install with the [storage-gcs] extra: the cloud worker PULLS the gs:// input and
PUSHES the
56     # gs:// outputs/done-marker through backend.storage (google-cloud-storage is lazy-
imported there).
57     # Without it the job dies at the first gs:// fetch with `ModuleNotFoundError:
google.cloud` ? the
58     # real-Cloud-Run failure the mock-client tests can't see (the bucket I/O is faked in
tests).
59     # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
60     # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
61     # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
62     # single-sourced in requirements-trackers.txt.
63     RUN python3 -m pip install --upgrade pip \
64         && python3 -m pip install -r requirements-trackers.txt \
65         && python3 -m pip install ".[storage-gcs]"
66
67     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
68     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
69     # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
70     # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
71     # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
72     ENV PYTHONUNBUFFERED=1 \
73         WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
74         WASB_REPO_DIR=/opt/models/WASB-SBDT \
75         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
76         RALLY_DETECTOR_IMPL=native
77
78     # The dispatcher (CloudRunCompute) supplies the per-execution args:
79     # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
80     ENTRYPOINT ["python3", "-m", "backend.worker"]
81

```

10. user (2026-07-03T19:48:43.497Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\docs\CODMAP.md:256:$
**manifest.json** (cache): {`version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done`}. Reuse only if ALL match + `version==CACHE_VERSION`.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-

```

```

indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\docs\CODE_MAP.md:271:-
`model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from
cfg["model"] at runtime in wasb_infer.run.)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:177:         frame_args = f"--
frames_out_dir {wsl_tilde_quote(frames_dir)}"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:151:     frames_in: int,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:174:         and manifest.get("frames_in") ==
frames_in
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:257: def build_windows(frames: list, frames_in:
int):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:258:     """Sliding windows of ``frames_in``
consecutive frame *paths* (the unit of GPU work
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:261:     Returns a list of windows, each a list
of ``frames_in`` consecutive paths
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:262:     ([frames[i:i+frames_in] for i in
range(len(frames) - frames_in + 1)]``). The
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:266:     return [frames[i : i + frames_in] for
i in range(len(frames) - frames_in + 1)]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:381:     in a ``SequentialFrameStore`` (sliding
buffer sized to ``window`` = ``frames_in``); on a
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:504:     frames_in =
int(cfg["model"]["frames_in"])
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:505:     input_wh =
(int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:506:     output_wh =
(int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:524:     if len(frames) < frames_in:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:526:         raise SystemExit(f"need >=
{frames_in} frames, found {len(frames)} in {where}")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:528:     # Sliding windows of `frames_in`
consecutive frames (the unit of GPU work + checkpoint).
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:530:     for window in build_windows(frames,

```

```

frames_in):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:550:         frames_in=frames_in,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:570:         "frames_in": frames_in,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:630:
window=frames_in,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:842:         "--frames_out_dir", help="where to
extract frames when --video is used"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:912:         args.video,
args.frames_out_dir or (args.video + "_frames")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\native_wasb_runner.py:272:         argv += ["--
frames_out_dir", frames_dir]
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_native_wasb_runner.py:292:         # Pin disk mode here (default is now
streaming) so this also covers the --frames_out_dir path.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_native_wasb_runner.py:307:         assert "--frames_out_dir" in argv and "--
stream-video" not in argv
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_native_wasb_runner.py:318:         assert "--stream-video" in argv and "--
frames_out_dir" not in argv
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_native_wasb_runner.py:334:         assert argv[argv.index("--frames_out_dir") +
1] == expected_frames
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_native_wasb_runner.py:343:         assert argv[argv.index("--frames_out_dir") +
1] == expected_frames
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_tracknet_runner.py:628:         assert "--frames_out_dir" not in cmd # no PNG
extraction on the fast path
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_tracknet_runner.py:635:         assert "--frames_out_dir" in cmd
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:132:         "frames_in": 3,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:147:         frames_in=3,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:166:         frames_in=3,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:174:         "frames_in": 3,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:204:         frames_in=3,

```

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\tests\test_wasb_cache.py:212:      "frames_in": 3,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\tests\test_wasb_cache.py:283:      # frames_in=3 -> 4 sliding windows, each 3 consecutive, overlapping by 2.
```

11. assistant (2026-07-03T19:48:47.380Z)

This is the critical finding. From `docs/CODE\_MAP.md:271`: `model=wasb` -> `frames\_in=3`, `frames\_out=3`, input 512x288. So the badminton model uses **frames_in=3** with sliding windows overlapping by 2 frames. Let me verify the exact overlap redundancy by examining the `ImageDataset.\_\_getitem\_\_` behavior more closely and the CODE_MAP for the WASB dataset.

12. user (2026-07-03T19:48:48.895Z)

```
245     `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
246
247     $ **jobs row** (`db.create_job` / `get_job` ? #16, PR-1): `{id:TEXT(uuid4.hex),
video_path, video_id?:TEXT(set once hashed), segmenter?, player_pool?:JSON list, max_frames?,
status:'queued'|"running"|"done"|"failed' (EXECUTION state ? pipeline verdict is
result["status"]), progress?:TEXT(stage label), error?:TEXT, result?:JSON(the former sync
/api/process response incl. reports), created_at, started_at?, finished_at?}`.
248
249     $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
250
251     $ **GT / golden CSVs** (TWO conventions ? do not conflate):
252     - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
253     - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
254
255     $ **det_raw.jsonl** (one window/line): `{ "w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
256     $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
257     $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv,
...}]}`; ? DB key = file MD5, manifest `video_id` is a label.
258
259     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
260
261     $ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
262
263     $ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
264
265     $ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges`
```



```

t}`, `validation.status`, `video_meta.fps_source`.
266
267 ---
268
269 ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
270 - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
271 - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
272 - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
273 - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
274 - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
275 - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
276
277 ---
278
279 ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
280 - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
281 - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@scripts/run_phase3_eval.py`.
282 - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
283
284 ---
285
286 ## 6. GOTCHAS (cross-cutting footguns)
287 - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
288 - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config['.indexing']` is `None` (the ASGI-import edge case).
289 - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.

```

13. user (2026-07-03T19:48:51.302Z)

```

130: - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{ }).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real

```

```

runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
271:- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from
cfg["model"] at runtime in wasb_infer.run.)
272:- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
274:- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)

```

14. assistant (2026-07-03T19:48:57.711Z)

I now have the critical facts. Let me confirm the exact `run\_tensor` semantics and how `imgs` is shaped (frames_in stacked per sample), by checking how `batch\_results` is indexed. The key question for redundancy is whether each window re-transforms all 3 frames independently.

Looking at the code: `imgs, \_hms, trans = batch[0], batch[1], batch[2]`, and `batch[-1]` is `img\_paths` shaped `[frame\_in][batch]`. `batch\_results[ib][ie]` is indexed by both batch item `ib` and element (frame-in) index `ie`. So each sample = one window of `frames\_in=3` frames, and the DataLoader/`\_\_getitem\_\_` transforms all 3 frames of that window.

Let me verify the overlap math and check the test that documents windowing overlap.

15. user (2026-07-03T19:49:00.895Z)

```

278         assert wi._frame_id(f"frame_{i:08d}.png") == i
279
280
281     def test_build_windows_matches_disk_logic():
282         frames = [wi.synthetic_frame_path(i) for i in range(6)]
283         # frames_in=3 -> 4 sliding windows, each 3 consecutive, overlapping by 2.
284         wins = wi.build_windows(frames, 3)
285         assert len(wins) == 4
286         assert wins[0] == frames[0:3]
287         assert wins[1] == frames[1:4]
288         assert wins[-1] == frames[3:6]
289
290
291     def test_build_windows_too_few_frames_is_empty():
292         assert wi.build_windows(["00000000.png", "00000001.png"], 3) == []
293
294
295     # ----- #
296     # streaming fast-path: SequentialFrameStore (sliding buffer; pure)
297     # ----- #
298     def _counting_reader():
299         """A fake forward-only decoder: returns ('frame', i) and records call order."""
300         calls = []
301
302         def reader(i):
303             calls.append(i)
304             return ("frame", i)
305
306         return reader, calls
307

```

16. user (2026-07-03T19:49:03.355Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-worktree\docs\CODE_MAP.md:272:-
`detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis)`.

```

```

`results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses
`detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md:92:    def
to(self, model_or_tensor): ...                # placement (cuda/mps/cpu)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md:131:So the GPU
is **cleanly abstracted** (stages call `ctx.to()/ctx.autocast()/ctx.knobs`, never `torch.cuda`
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:269:class SequentialFrameStore:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:381:    in a ``SequentialFrameStore`` (sliding
buffer sized to ``window`` = ``frames_in``); on a
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:387:    store = SequentialFrameStore(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:403:class _PrefetchIter:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:637:        _PrefetchIter(loader,
depth=prefetch_batches)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\backend\pipeline\detectors\wasb_infer.py:646:        batch_results, _ =
detector.run_tensor(imgs, trans)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:296:# streaming fast-path: SequentialFrameStore (sliding
buffer; pure)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:315:    store = wi.SequentialFrameStore(reader, total=5,
window=3)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:325:    store = wi.SequentialFrameStore(reader, total=100,
window=3)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:337:    store = wi.SequentialFrameStore(reader, total=10,
window=3)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:348:    store = wi.SequentialFrameStore(reader, total=10,
window=3, start_index=4)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:358:    store = wi.SequentialFrameStore(reader, total=3,
window=3)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:381:    store = wi.SequentialFrameStore(loader, total=4,
window=1)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\scratchpad\fix-
worktree\tests\test_wasb_cache.py:408:    store = wi.SequentialFrameStore(loader, total=10,
window=1, start_index=5)

```

I now have all the facts I need. Let me confirm the key redundancy insight by reasoning carefully about the data flow.

****The overlap redundancy is confirmed and severe.**** With ``frames_in=3`` and stride-1 sliding windows:

- ``build_windows`` produces windows ``[0,1,2], [1,2,3], [2,3,4], ...`` ? N-2 windows for N frames.
- Each window is a ``sample`` fed to ``ImageDataset.__getitem__``, which reads ****all 3 frames**** of that window via ``read_image(path)`` ? PIL ? resize to 512x288 ? ToTensor ? ImageNet normalize.
- Frame ``k`` (for interior frames) appears in exactly 3 windows: as position 2 of window ``k-2``, position 1 of window ``k-1``, position 0 of window ``k``. So ****every interior frame is decoded + transformed 3 times**** (= ``frames_in`` times).

Let me verify one more thing: whether ``run_tensor`` even benefits from a large batch, given the HRNet forward and whether the detector moves tensors to GPU. Let me check the ``SequentialFrameStore`` decode-once guarantee vs the transform redundancy (decode is cached, transform is not).

18. user (2026-07-03T19:49:13.847Z)

```
308
309     def test_store_reads_each_frame_once_in_order_for_sliding_windows():
310         # The detector's real access pattern: ImageDataset reads window i's frames in order
311         # (i, i+1, i+2), and samples are requested i=0,1,2,... So the global read sequence
312         # DIPS BACK by window-1 at each boundary: 0,1,2, 1,2,3, 2,3,4. Every source frame
313         # must still be decoded exactly once (the whole point of the in-memory buffer).
314         reader, calls = _counting_reader()
315         store = wi.SequentialFrameStore(reader, total=5, window=3)
316         order = [0, 1, 2, 1, 2, 3, 2, 3, 4]
317         got = [store.get(i) for i in order]
318         assert got == [{"frame", i} for i in order]
319         assert calls == [0, 1, 2, 3, 4] # decoded once each, in order
320
321
322     def test_store_evicts_outside_window_bounded_memory():
323         # window=3 -> buffer holds at most 3 frames; frames older than max_seen-2 are
324         # dropped.
325         reader, _ = _counting_reader()
326         store = wi.SequentialFrameStore(reader, total=100, window=3)
327         for i in (0, 1, 2, 3, 4):
328             store.get(i)
329         assert len(store._buf) <= 3
330         # frame 1 is now below the floor (max_seen=4, floor=2) -> evicted, re-request
331         # raises.
332         with pytest.raises(KeyError):
333             store.get(1)
334
335     def test_store_keeps_window_minus_one_lookback():
336         # The legal dip-back of exactly window-1 must NOT raise (it's the real access
337         # pattern).
338         reader, _ = _counting_reader()
339         store = wi.SequentialFrameStore(reader, total=10, window=3)
340         store.get(0)
341         store.get(1)
342         store.get(2) # max_seen=2, floor=0 -> frames 0,1,2 all retained
343         assert store.get(1) == ("frame", 1) # window boundary look-back is fine
344         assert store.get(2) == ("frame", 2)
345
346     def test_store_resume_starts_at_start_index_without_decoding_prefix():
347         # On a resume the first needed frame is `start_index`; the prefix must NOT be
348         # decoded.
349         reader, calls = _counting_reader()
350         store = wi.SequentialFrameStore(reader, total=10, window=3, start_index=4)
351         store.get(4)
```

```
350         store.get(5)
351         assert calls == [4, 5] # frames 0..3 skipped entirely (seek, not decode)
352         with pytest.raises(KeyError): # below the resume floor -> never available
```

19. assistant (2026-07-03T19:49:21.021Z)

This confirms the critical asymmetry precisely. The `SequentialFrameStore` guarantees each source frame is **decoded** exactly once (the `reader(i)` / `cv2` decode is cached). But that cached frame is a BGR uint8 ndarray. The expensive per-window transform happens *downstream* of the store, inside `ImageDataset.__getitem__`, which runs on the frame returned by `_bgr_to_pil_rgb(store.get(idx))` ? so:

- `_bgr_to_pil_rgb` (the `cv2.cvtColor` BGR→RGB + `Image.fromarray`) runs **once per (window, position)** = `frames_in` × per frame.
- The resize to 512×288 + `ToTensor` + ImageNet normalize inside `__getitem__` runs **once per (window, position)** = `frames_in` × per frame.

So the decode is deduplicated, but the color-convert + resize + normalize (the bulk of the CPU cost that the `_PrefetchIter` thread is trying to overlap) is done **3× redundantly** for every interior frame. This is the second-order finding.

I have everything needed to write the structured finding.

20. user (2026-07-03T19:50:00.508Z)

Structured output provided successfully